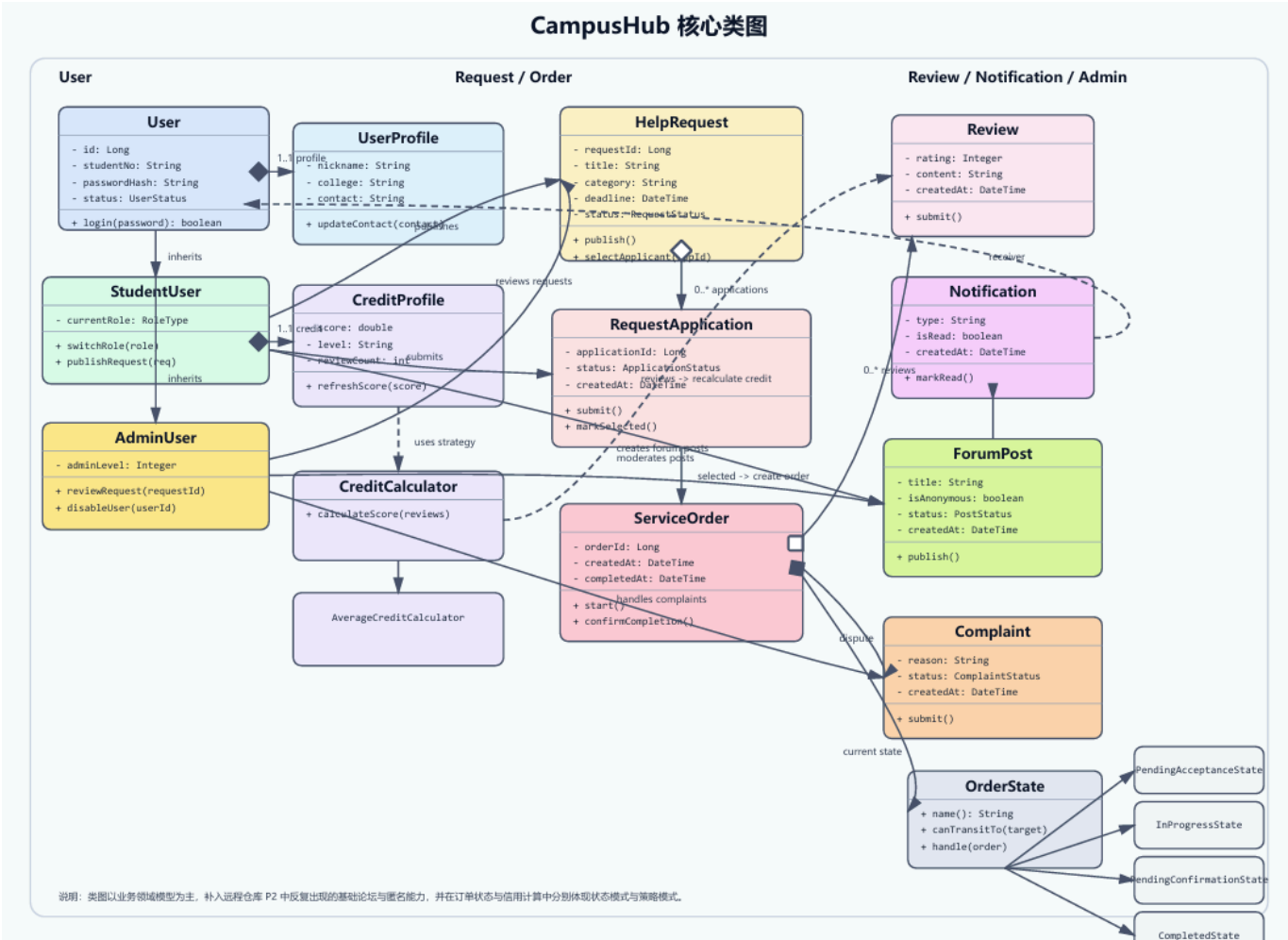


项目设计文档（内容整合）

团队：第67组 日期：2026/5/7

一、类图绘制



关键类说明

类名	职责说明	关键属性	关键方法
User	所有平台用户的抽象父类，提供账号、状态和基础行为。	id studentNo passwordHash status	login() logout() updateProfile()
StudentUser	学生用户，既可以作为需求方也可以作为服务方。	currentRole creditScore	switchRole() publishRequest() applyForRequest()
AdminUser	管理员用户，负责治理和审查。	adminLevel	reviewRequest() disableUser() resolveComplaint()

类名	职责说明	关键属性	关键方法
HelpRequest	平台互助需求实体。	title description category deadline status	publish() edit() close() selectApplicant()
RequestApplication	服务方对需求发起的接单申请。	applicationId message status	submit() withdraw() markSelected() markRejected()
ServiceOrder	需求确认后的正式订单，承载状态流转。	orderId createdAt completedAt	start() submitCompletion() confirmCompletion() rejectCompletion()
Review	订单完成后双向评价记录。	rating content createdAt	submit()
Notification	站内通知对象。	type content isRead	markRead()
ForumPost	基础论坛发帖对象，用于补充校内交流与信息发布。	title content isAnonymous status	publish() edit() close()
Complaint	订单争议与投诉记录。	reason status	submit() closeCase()
OrderState	订单状态抽象接口，封装状态迁移规则。	-	canTransitTo() handle()
CreditCalculator	信用分计算策略接口。	-	calculateScore()

关系说明

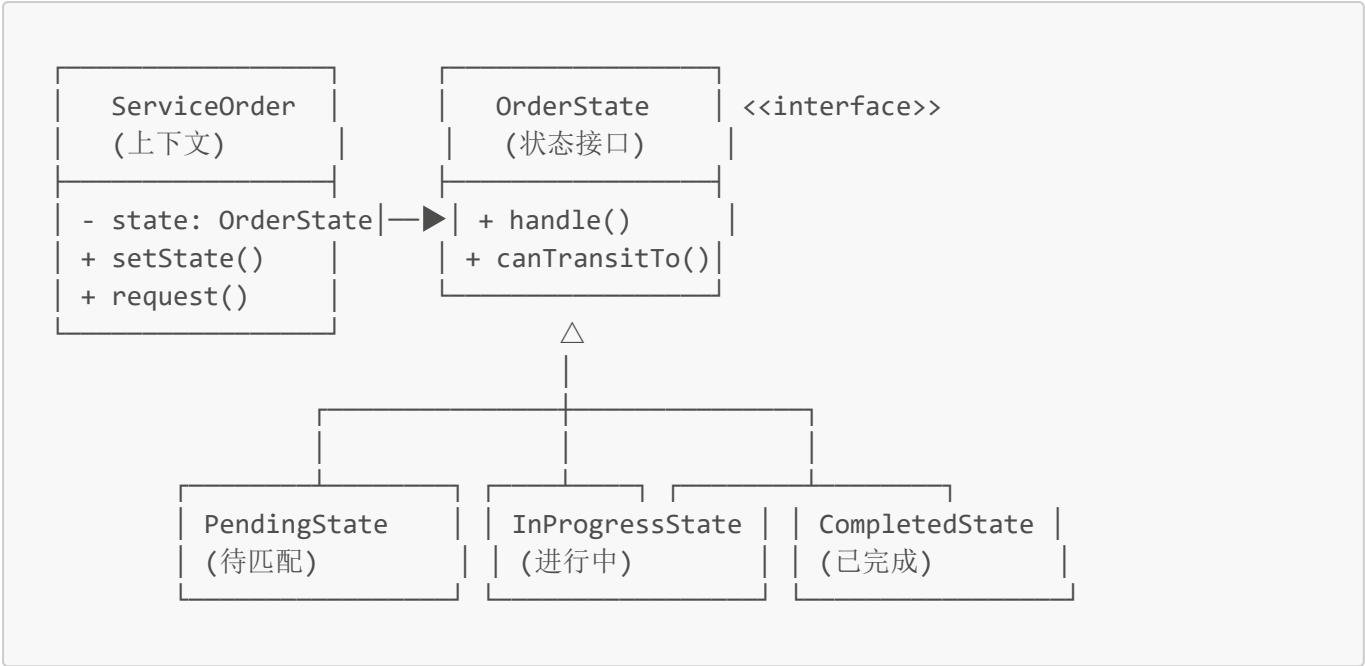
- StudentUser 和 AdminUser 继承自 User。
- User 与 UserProfile 是组合关系，StudentUser 与 CreditProfile 是组合关系。
- HelpRequest 聚合多个 RequestApplication，确认服务方后生成 ServiceOrder。
- ServiceOrder 关联需求方与服务方两个 StudentUser，并在完成后产生 Review。
- StudentUser 可以创建 ForumPost，用于实现远程仓库文档中提到的“基础论坛”能力。
- AdminUser 可以审核 HelpRequest 与 ForumPost，体现管理后台对内容治理的职责。
- Complaint 关联 ServiceOrder，用于记录争议与后台处理过程。
- ServiceOrder 持有 OrderState，CreditProfile 依赖 CreditCalculator。

设计模式说明

一、状态模式（State Pattern）

状态模式是一种行为型设计模式，它允许一个对象在其内部状态改变时改变其行为。该模式将状态相关的行为封装到独立的状态类中，使得原本庞大的条件分支语句被分散到各个状态类中，从而遵循开闭原则和单一职责原则。

模式结构

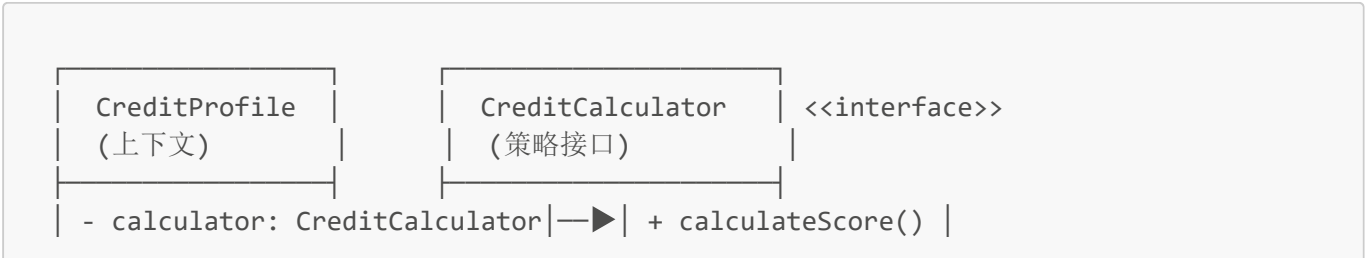


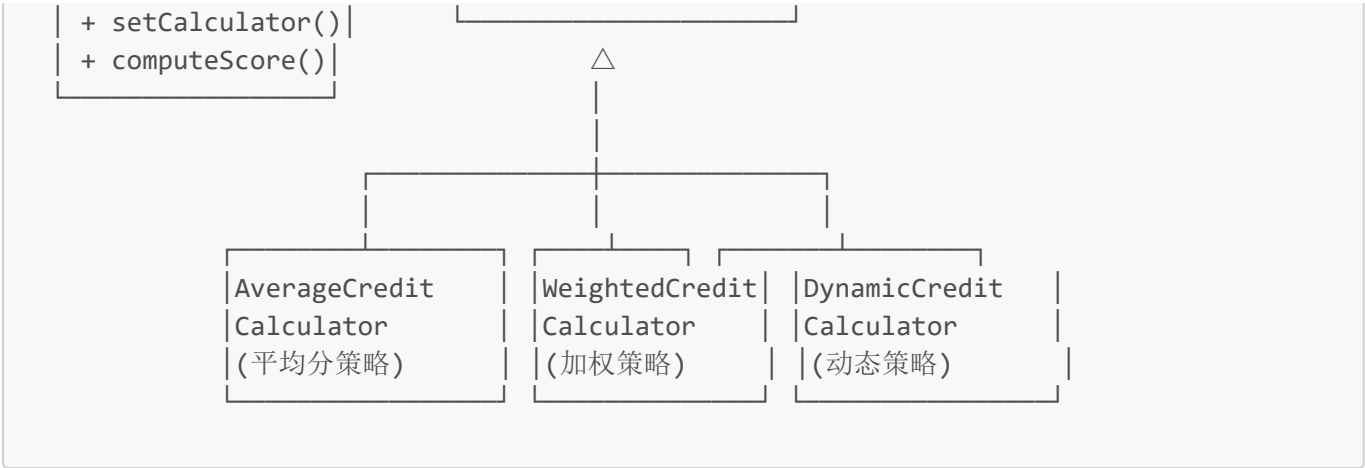
使用状态模式的理由

理由	详细说明
订单状态复杂多变	服务订单可能经历：待匹配 → 进行中 → 待确认 → 已完成 → 已评价，还可能包含争议、取消等异常状态。状态之间存在复杂的迁移规则。
避免巨型条件分支	不使用状态模式时，需要在 <code>ServiceOrder</code> 类中写满 <code>if-else</code> 或 <code>switch-case</code> 判断当前状态来决定允许什么操作。状态模式将每个状态的行为内聚到单独类中。
状态迁移规则集中管理	<code>canTransitTo()</code> 方法明确规定了从当前状态可以转移到哪些目标状态，规则一目了然，便于维护和修改。
符合开闭原则	如果需要新增一种状态（如 <code>DisputedState</code> 争议状态），只需新增一个类实现 <code>OrderState</code> 接口，无需修改现有状态类或订单类。
订单操作行为与状态解耦	<code>handle()</code> 方法封装了该状态下执行操作的完整逻辑，订单类无须关心具体实现，只需委托给当前状态对象。
真实业务场景匹配	校园互助场景中，订单状态的流转有明确的业务规则（如：只有进行中的订单才能申请完成），状态模式天然映射这种规则。

二、策略模式 (Strategy Pattern) 策略模式是一种行为型设计模式，它定义了一系列算法（策略），将每个算法封装起来，并使它们可以相互替换。策略模式让算法的变化独立于使用算法的客户端，从而遵循开闭原则。

模式结构





使用策略模式的理由

理由	详细说明
信用计算规则可能变化	产品初期可能采用简单的平均分策略；随着业务发展，可能需要引入加权策略（考虑时间衰减、订单金额、完成率等）。策略模式支持平滑切换。
A/B 测试需求	平台可能需要同时测试多种信用计算算法对用户行为的影响。策略模式允许动态替换算法，便于进行对比实验。
不同用户群体适用不同规则	新用户可能没有足够评价数据，需要特殊的初始信用策略；高信誉用户可能需要更精细的计算模型。策略模式可动态选择适合的算法。
避免 CreditProfile 类臃肿	如果将多种信用计算逻辑全部写在 CreditProfile 类中，会导致该类过于庞大且难以维护。策略模式将算法独立出来。
符合开闭原则	新增一种信用计算策略（如引入机器学习模型预测信用）时，只需新增实现类，无需修改 CreditProfile 或其他现有策略。
便于单元测试	每种策略可以独立进行单元测试，验证计算逻辑的正确性，而不依赖整个 CreditProfile 上下文。
业务规则与计算逻辑分离	信用分作为平台信任体系的核心，其计算逻辑应当清晰、可追溯。策略模式将计算逻辑封装在独立的策略类中，便于业务人员审查和调整。

三、两种设计模式的对比

维度	状态模式	策略模式
核心关注点	对象状态的变化及其对应的行为变化	对象算法的可替换性
状态/策略数量	通常是有限且预知的（订单生命周期状态）	可以无限扩展（多种信用计算算法）
状态/策略之间关系	状态之间有关联（定义了迁移规则）	策略之间相互独立，无关联
上下文角色	ServiceOrder 持有当前状态，状态会变化	CreditProfile 持有某个策略，策略可替换
典型应用场景	工作流、订单状态机、生命周期管理	算法族、计算规则、数据校验规则

维度	状态模式	策略模式
在 CampusHub 中的角色	管理订单从发布到完成的完整流转逻辑	管理信用分计算的多样化算法

在项目设计中：

- **状态模式**被用于 `ServiceOrder` 订单状态管理，因为订单生命周期有明确的状态节点和迁移规则，状态模式能够优雅地封装状态流转逻辑，避免巨型条件分支，同时便于后续扩展新状态（如争议状态）。
- **策略模式**被用于 `CreditCalculator` 信用分计算，因为信用规则可能随业务发展而调整（平均分 → 加权分 → 动态分），策略模式实现了算法与上下文的解耦，支持运行时切换策略，也为未来的 A/B 测试和差异化规则提供了架构基础。

两种设计模式的共同价值在于：**提高代码的可维护性、可扩展性，降低耦合度**，这与 P2 架构文档中强调的“模块化”和“清晰的边界划分”是一脉相承的。

二、API 设计

接口汇总表

序号	接口	方法	说明
1	<code>POST /users/add</code>	POST	用户注册
2	<code>POST /users/token</code>	POST	用户登录
3	<code>POST /needs/add</code>	POST	发布需求
4	<code>POST /needs/list</code>	POST	浏览需求列表（含筛选）
5	<code>POST /orders/add</code>	POST	接单
6	<code>GET /orders/{orderId}</code>	GET	查看订单详情
7	<code>POST /reviews/add</code>	POST	提交评价
8	<code>PUT /orders/{orderId}/status</code>	PUT	更新订单状态
9	<code>POST /users/{userId}/orders/list</code>	POST	查看用户的订单列表
10	<code>PUT /demands/{demandId}/cancel</code>	PUT	取消未接单的需求
11	<code>GET /users/{userId}</code>	GET	获取用户信息
12	<code>PUT /users/password/reset</code>	PUT	重置密码
13	<code>PATCH /users/{userId}</code>	PATCH	更新用户资料
14	<code>POST /users/{userId}/reviews/list</code>	POST	查看用户收到的评价列表
15	<code>POST /users/{userId}/needs/list</code>	POST	获取我发布的需求列表
16	<code>POST /orders/list</code>	POST	获取我的订单列表
17	<code>PUT /orders/{orderId}/complete</code>	PUT	申请完成订单

序号	接口	方法	说明
18	GET /orders/{orderId}/review	GET	获取订单评价详情
29	POST /notifications/list	POST	获取通知列表

请求失败响应格式

```
{
  "code": ${errorCode},
  "message": ${message},
  "success": false,
}
```

1. 用户注册

接口：POST /users/add

请求参数

参数	类型	必填	说明
phone	num	是	11位数字
password	string	是	密码加盐得到的字符串

成功示例

```
{
  "code": 200,
  "msg": "账号注册成功",
  "success": true,
  "data": {
    "userId": 100001,
    "userName": "张三",
  }
}
```

错误码说明

错误码	说明
400	手机号缺失 / 密码缺失
403	手机号已使用
500	服务器内部错误

2. 用户登录

请求参数

参数	类型	必填	说明
phone	num	是	11位数字
password	string	是	密码加盐得到的字符串

成功示例

```
{
  "code": 200,
  "msg": "账号登录成功",
  "success": true,
  "data": {
    "userId": 100001,
    "userName": "张三",
    "token": "eyJhbGciOiJ0eS5jbGciOiJ0eS5jbGciOiJ0eS5jbGci",
    "avatar": "https://example.com/avatar.png",
    "role": "user",
  }
}
```

错误码说明	错误码	说明
	400	手机号缺失 / 密码缺失
	403	账号不存在 / 密码错误
	500	因为服务器内部原因导致登录失败

3. 发布需求

请求参数

参数	类型	必填	说明
title	string	是	标题，最多30字
category	枚举	是	分类： cleaning/repair/moving/tutoring/other
description	string	是	需求详细描述
budget	integer	是	预算金额，单位：元
address	string	是	服务地址
expectedTime	DateTime	是	期望服务时间
contactPhone	DateTime	是	联系电话

成功响应示例

```
{
  "code": 200,
  "message": "成功创建需求",
  "success": true,
  "data": {
    "Id": 1000,
    "status": "pending",
    "createdAt": "2025-01-15T10:30:00Z"
  }
}
```

错误码说明

错误码	场景
400	必填字段缺失或参数格式错误
401	Token无效或过期
500	因为服务器内部问题发布失败

4. 浏览需求列表（含筛选）

接口：POST /needs/list

请求参数

参数	类型	必填	说明
category	string	否	分类筛选，多个用逗号分隔
status	string	否	pending/accepted/completed，默认pending
minBudget	integer	否	最低预算
maxBudget	integer	否	最高预算
radius	integer	否	距离范围，默认为最大值
sortBy	string	否	排序字段：createdAt/budget/distance，默认createdAt
order	string	否	asc/desc，默认desc
page	integer	否	页码，默认1
pageSize	integer	否	每页数量，默认20，最大100

成功响应

```
{
  "code": 200,
  "msg": "查询成功",
}
```

```
"success": true,
"data": {
  "total": 153,
  "page": 1,
  "pageSize": 20,
  "list": [
    {...},{...},...
  ]
}
```

错误码说明

错误码	场景
401	分页参数错误 / 筛选无效
500	因为服务器内部原因查询失败

5. 接单

接口：POST /orders/add

请求体参数

参数	类型	必填	说明
needId	integer	是	需求ID

成功响应

```
{
  "code": 200,
  "message": "成功接取需求并创建订单",
  "success": true,
  "data": {
    "demandId": 300001,
    "status": "accepted",
    "acceptedBy": 100002,
    "acceptedAt": "2025-01-15T11:00:00Z"
  }
}
```

错误码说明

错误码	场景
401	demandId无效
403	接了自己的需求 / 用户不可接单（被禁用）

错误码	场景
404	需求不存在
500	因为服务器内部原因导致接单失败

6. 查看订单详情

接口：GET /orders/{orderId}

URL参数

参数	类型	必填	说明
orderId	integer	是	订单ID（接单后生成）

响应

成功响应

```
{
  "code": 200,
  "message": "success",
  "data": {
    "orderId": 500001,
    "demand": {
      "needId": 300001,
      "title": "水管漏水需要维修",
      "description": "厨房水龙头下方水管漏水...",
      "category": "repair",
      "budget": 20000,
      "address": "朝阳区XX小区3号楼101",
      "contactPhone": "138xxxxxxxx",
      "expectedTime": "2025-01-16T14:00:00Z"
    },
    "status": "accepted",
    "publisher": {
      "userId": 100001,
      "nickname": "张三",
      "phone": "138***0000",
    },
    "acceptor": {
      "userId": 100002,
      "nickname": "李师傅",
      "phone": "139***1111",
    },
  },
}
```

错误码说明

错误码	场景
401	登录状态未认证
403	无权查看此订单（非发布者或服务方）
404	订单不存在
500	因为服务器内部原因查询失败

7. 提交评价

接口：POST /reviews/add

请求参数（Body）

参数	类型	必填	说明
score	integer	是	评分：1~5
content	string	否	评价内容，最多200字
orderId	integer	是	订单ID
userId	integer	是	用户ID

成功响应

```
{
  "code": 200,
  "message": "success",
  "success": true,
  "data": {
    "evaluationId": 800001,
    "orderId": 500001,
    "score": 5,
    "content": "师傅很专业，处理得很快",
    "createdAt": "2025-01-17T15:30:00Z",
  }
}
```

错误码说明

错误码	场景
401	用户登录状态未认证
403	score不在1-5之间 / 非发布者 / 订单状态不是completed / 已评价过
404	订单不存在
500	因为服务器内部原因提交评价失败

8. 更新订单状态

接口：PUT /orders/{orderId}/status

URL参数

参数	类型	必填	说明
orderId	integer	是	订单ID

请求参数 (Body)

参数	类型	必填	说明
action	string	是	start (开始服务) / complete (完成服务) / cancel (取消订单)

成功响应

```
{
  "code": 200,
  "message": "成功更新订单状态",
  "success": true,
  "data": {
    "orderId": 500001,
    "status": "started",
    "updatedAt": "2025-01-16T14:00:00Z"
  }
}
```

错误码说明

错误码	场景
401	用户登录状态未认证
403	action无效，如订单从completed到start
404	订单不存在
500	服务器内部错误

9. 查看用户的订单列表

接口：POST /users/{userId}/orders/list

URL参数

参数	类型	必填	说明
userId	integer	是	用户ID

请求参数 (Body)

参数	类型	必填	说明
role	string	否	<code>publisher</code> (作为发布者) / <code>acceptor</code> (作为服务方), 不传则返回全部
status	string	否	订单状态: <code>pending/accepted/started/completed/cancelled</code>
page	integer	否	页码, 默认1
pageSize	integer	否	每页数量, 默认20, 最大100

成功响应

```
{
  "code": 200,
  "message": "成功查询订单列表",
  "success": true,
  "data": {
    "total": 25,
    "page": 1,
    "pageSize": 20,
    "list": [
      {...}, {...}, ...
    ]
  }
}
```

错误码说明

错误码	场景
401	用户登录状态未认证
404	用户不存在
500	服务器内部错误

10. 取消未接单的需求

接口: `PUT /needs/{demandId}/cancel`

URL参数

参数	类型	必填	说明
demandId	integer	是	需求ID

参数	类型	必填	说明
reason	string	否	取消原因, 最多100字

成功响应

```
{
  "code": 200,
  "message": "成功取消需求",
  "success": true,
  "data": {
    "demandId": 300001,
    "status": "cancelled",
    "cancelledAt": "2025-01-15T12:00:00Z"
  }
}
```

错误码说明

错误码	场景
401	用户登录状态未认证
403	无法取消（非发布者/已接单）
500	服务器内部错误

11. 获取当前用户信息

接口：GET /users/{userId}

URL参数

参数	类型	必填	说明
userId	integer	是	用户ID（传入当前用户ID即为自己）

成功响应

```
{
  "code": 200,
  "message": "成功查询当前用户信息",
  "success": true,
  "data": {
    "userId": 100001,
    "phone": "13812345678",
    "nickname": "张三",
    "avatar": "https://cdn.example.com/avatar/100001.jpg",
    "role": "both",
    "totalOrders": 42,
    "completedOrders": 38,
    "createdAt": "2024-01-01T00:00:00Z"
  }
}
```

错误码说明

错误码	场景
401	用户登录状态未认证
404	用户不存在
500	服务器内部错误

12. 重置密码（应该用不上）

接口：PUT /users/password/reset

请求参数（Body）

参数	类型	必填	说明
phone	string	是	手机号
code	string	是	短信验证码
newPassword	string	是	新密码，6~20位

成功响应

```
{
  "code": 200,
  "message": "密码重置成功",
  "success": true,
  "data": null
}
```

错误码说明

错误码	场景
400	参数格式错误（手机号/密码不符合规则）
401	验证码错误或已过期
404	手机号不存在
500	服务器内部错误

13. 更新用户资料

接口：PATCH /users/{userId}

URL参数

参数	类型	必填	说明
userId	integer	是	用户ID

请求参数 (Body)

参数	类型	必填	说明
nickname	string	否	昵称, 2~20字
avatar	string	否	头像URL
email	string	否	邮箱, 格式如: xxx@example.com

成功响应

```
{
  "code": 200,
  "message": "更新成功",
  "success": true,
  "data": {
    "nickname": "新昵称",
    "avatar": "https://cdn.example.com/avatar/100001.jpg",
    "updatedAt": "2025-01-18T10:00:00Z"
  }
}
```

错误码说明

错误码	场景
400	参数格式错误
401	用户登录状态未认证
403	无权修改他人资料
404	用户不存在
500	服务器内部错误

14. 查看用户收到的评价列表

接口： `POST /users/{userId}/reviews/list`

URL参数

参数	类型	必填	说明
userId	integer	是	用户ID

请求参数 (Body)

参数	类型	必填	说明
score	integer	否	按评分筛选：1~5
page	integer	否	页码，默认1
pageSize	integer	否	每页数量，默认20，最大100

...

成功响应

```
{
  "code": 200,
  "message": "success",
  "success": true,
  "data": {
    "total": 87,
    "averageScore": 4.85,
    "scoreDistribution": {
      "5": 72,
      "4": 10,
      "3": 3,
      "2": 1,
      "1": 1
    },
    "list": [
      {...},{...},{...},...
    ]
  }
}
```

错误码说明

错误码	场景
400	参数格式错误
404	用户不存在
500	服务器内部错误

15. 获取我发布的需求列表

接口：POST /users/{userId}/needs/list

URL参数

参数	类型	必填	说明
userId	integer	是	用户ID

请求参数 (Body)

参数	类型	必填	说明
status	string	否	pending/accepted/completed/cancelled
page	integer	否	页码，默认1
pageSize	integer	否	每页数量，默认20，最大100

成功响应

```
{
  "code": 200,
  "message": "success",
  "success": true,
  "data": {
    "total": 12,
    "page": 1,
    "pageSize": 20,
    "list": [
      {...},{...},...
    ]
  }
}
```

错误码说明

错误码	场景
401	用户登录状态未认证
403	无权查看他人需求列表
404	用户不存在
500	服务器内部错误

16. 获取我的订单列表（作为参与方）

接口：POST /orders/list

请求参数 (Body)

参数	类型	必填	说明
role	string	否	publisher（作为发布者）/ acceptor（作为服务方），不传则返回全部
status	string	否	订单状态：pending/accepted/started/completed/cancelled
page	integer	否	页码，默认1
pageSize	integer	否	每页数量，默认20，最大100

成功响应

```
{
  "code": 200,
  "message": "success",
  "success": true,
  "data": {
    "total": 3,
    "page": 1,
    "pageSize": 20,
    "list": [
      {...},{...},....
    ]
  }
}
```

错误码说明

错误码	场景
401	用户登录状态未认证
403	无权查看他人订单列表
500	服务器内部错误

17. 申请完成订单

接口：PUT /orders/{orderId}/complete

URL参数

参数	类型	必填	说明
orderId	integer	是	订单ID

请求参数 (Body)

参数	类型	必填	说明
completionProof	string	否	完成凭证 (图片URL)

成功响应

```
{
  "code": 200,
  "message": "成功申请完成订单",
  "success": true,
  "data": {
    "orderId": 500001,
  }
}
```

```
    "status": "completing",
    "needConfirm": true,
    "updatedAt": "2025-01-16T16:30:00Z"
  }
}
```

错误码说明

错误码	场景
401	用户登录状态未认证
403	无权操作（非服务方）/ 订单状态不是started，无法申请完成
404	订单不存在
500	服务器内部错误

18. 获取订单评价详情

接口：GET /orders/{orderId}/review

URL参数

参数	类型	必填	说明
orderId	integer	是	订单ID

成功响应

```
{
  "code": 200,
  "message": "success",
  "success": true,
  "data": {
    "evaluationId": 800001,
    "orderId": 500001,
    "score": 5,
    "content": "师傅很专业，处理得很快",
    "tags": ["准时", "专业"],
    "fromUser": {
      "userId": 100001,
      "nickname": "张三",
      "avatar": "https://..."
    },
    "toUser": {
      "userId": 100002,
      "nickname": "李师傅",
      "avatar": "https://..."
    },
    "createdAt": "2025-01-17T15:30:00Z"
  }
}
```

```
    }  
  }  
}
```

错误码说明

错误码	场景
401	用户登录状态未认证
403	无权查看（非订单参与者）
404	订单不存在 / 该订单暂无评价
500	服务器内部错误

19. 获取通知列表

接口： `POST /notifications/list`

请求参数（Body）

参数	类型	必填	说明
isRead	boolean	否	筛选已读/未读，不传返回全部
type	string	否	通知类型： <code>order/system/review</code>
page	integer	否	页码，默认1
pageSize	integer	否	每页数量，默认20，最大100

成功响应

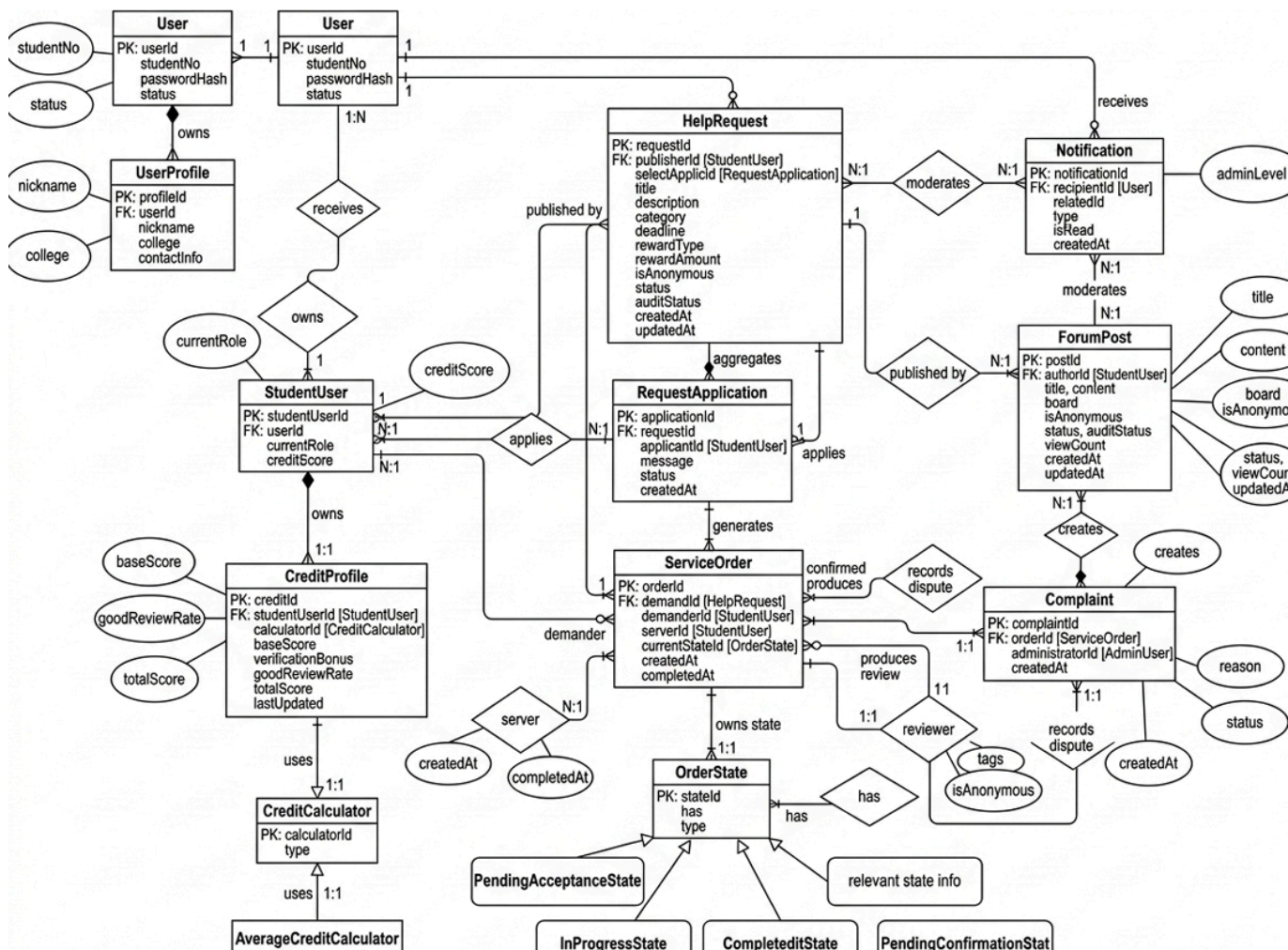
```
{  
  "code": 200,  
  "message": "success",  
  "success": true,  
  "data": {  
    "total": 8,  
    "unreadCount": 5,  
    "page": 1,  
    "pageSize": 20,  
    "list": [  
      {...},{...},...  
    ]  
  }  
}
```

错误码说明

错误码	场景
401	用户登录状态未认证
403	无权查看（非订单参与者）
500	服务器内部错误

三，E-R图设计

核心E-R图如下



核心实体及其关系说明

1. 用户与权限模块 (User & Profiles)

- User (用户): 基础实体, 包含学号 (studentNo) 和加密密码。
- StudentUser (学生用户): 继承自 User, 拥有信用分 (creditScore) 和当前角色 (需求方/服务方)。
- AdminUser (管理员): 继承自 User, 负责审核内容 (需求、帖子) 和处理投诉。
- UserProfile (个人资料): 与 User 为 1:1 组合关系, 存储昵称、学院、联系方式等非核心属性。
- CreditProfile (信用档案): 与 StudentUser 为 1:1 组合关系, 记录信用分明细, 由 CreditCalculator 策略类计算。

2. 需求与申请模块 (HelpRequest & Application)

- HelpRequest (需求/任务)：由 StudentUser 发布。
- 包含标题、描述、分类、赏金 (rewardAmount) 和截止日期。
- 状态机：待审核、招募中、已匹配、已完成、已关闭。
- RequestApplication (接单申请)：多个 StudentUser 可以对同一个 HelpRequest 发起申请 (1:N 聚合关系)。

3. 订单与交易模块 (ServiceOrder)

- ServiceOrder (服务订单)：当发布者从申请者中选定服务方后生成。
- 关联关系：关联一个 HelpRequest，并同时连接两个 StudentUser (需求方 Demander 和 服务方 Server)。
- OrderState (订单状态)：采用状态模式，包含待处理 (Pending)、进行中 (InProgress)、待确认 (WaitConfirm) 等状态。

4. 评价、通知与互动 (Interaction)

- Review (评价)：订单完成后产生。双向评价：每笔 ServiceOrder 对应两条评价记录 (需求方评服务方，服务方评需求方)。
- Notification (通知)：系统向 User 发送的异步消息，关联相关的实体 ID (如订单 ID 或需求 ID)。
- ForumPost (论坛帖子)：StudentUser 发布的交流信息，可由 AdminUser 审核。
- Complaint (投诉/争议)：关联特定的 ServiceOrder，由 AdminUser 介入处理。

关键约束逻辑

- 信用分驱动：StudentUser 的 creditScore 会影响其接单权限，由CreditCalculator 根据 Review 的好评率动态更新。
- 状态流转限制：ServiceOrder 的状态变迁受 OrderState 子类控制，例如只有在 InProgress 状态下才能提交完成申请。
- 角色隔离：虽然 User 是统一的，但在 API 层面通过 currentRole 区分当前用户是以“需求方”还是“服务方”身份进行操作。

四，数据库表结构与建表语句

1. 用户基础表 (users)

字段名	类型	约束	说明
user_id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	用户唯一标识，内部使用，不对外暴露
student_no	VARCHAR(20)	NOT NULL, UNIQUE	学号，作为登录凭证，唯一标识学生身份
password_hash	VARCHAR(255)	NOT NULL	存储 Argon2/BCrypt 加密后的密码哈希值，长度预留足够空间
status	TINYINT	DEFAULT 1	账号状态：1-正常，0-禁用（冻结/毕业注销）
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	账号创建时间

字段名	类型	约束	说明
updated_at	TIMESTAMP	ON UPDATE CURRENT_TIMESTAMP	最后更新时间，用于审计追踪

建表语句

```
-- 1. 用户基础表（实现类表继承，存储通用凭证）
CREATE TABLE `users` (
  `user_id` BIGINT PRIMARY KEY AUTO_INCREMENT,
  `student_no` VARCHAR(20) NOT NULL UNIQUE COMMENT '学号，唯一标识',
  `password_hash` VARCHAR(255) NOT NULL COMMENT 'Argon2或BCrypt加密后的哈希值',
  `status` TINYINT DEFAULT 1 COMMENT '1:正常, 0:禁用',
  `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  `updated_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  INDEX `idx_student_no` (`student_no`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

2. 学生扩展表（student_users）

字段名	类型	约束	说明
student_user_id	BIGINT	PRIMARY KEY	学生扩展记录ID，与 user_id 独立，便于分表扩展
user_id	BIGINT	NOT NULL, FOREIGN KEY	关联 users.user_id，1对1关系
nickname	VARCHAR(50)		用户昵称，可自定义，允许为空
college	VARCHAR(100)		所在院系，用于筛选同院系互助场景
contact_info_enc	VARBINARY(255)		联系方式（手机号/邮箱等），使用 AES 等算法加密存储，保护隐私
credit_score	DECIMAL(5,2)	DEFAULT 100.00	信用分，范围 0~100，用于信用体系建设（违约扣分、优质履约加分）
current_role	ENUM	DEFAULT 'DEMANDER'	当前角色：DEMANDER-需求方，SERVER-服务方，用户可切换

建表语句

```
-- 2. 学生扩展表（存储业务属性）
CREATE TABLE `student_users` (
  `student_user_id` BIGINT PRIMARY KEY,
  `user_id` BIGINT NOT NULL,
  `nickname` VARCHAR(50),
  `college` VARCHAR(100),
  `contact_info_enc` VARBINARY(255) COMMENT '加密存储的联系方式',
```

```
`credit_score` DECIMAL(5,2) DEFAULT 100.00,
`current_role` ENUM('DEMANDER', 'SERVER') DEFAULT 'DEMANDER',
FOREIGN KEY (`user_id`) REFERENCES `users`(`user_id`),
INDEX `idx_user_id` (`user_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

3. 需求发布表 (`help_requests`)

字段名	类型	约束	说明
<code>request_id</code>	BIGINT	PRIMARY KEY, AUTO_INCREMENT	需求唯一标识
<code>publisher_id</code>	BIGINT	NOT NULL, FOREIGN KEY	发布者ID, 关联 <code>student_users.student_user_id</code>
<code>category_id</code>	INT	NOT NULL	需求分类ID, 关联分类配置表 (待补充), 如编程、设计、跑腿等
<code>title</code>	VARCHAR(100)	NOT NULL	需求标题, 简洁概括
<code>description</code>	TEXT	NOT NULL	详细描述, 支持富文本或纯文本
<code>reward_amount</code>	DECIMAL(10,2)	DEFAULT 0.00	悬赏金额, 0表示无偿互助
<code>deadline</code>	DATETIME	NOT NULL	招募截止时间, 超时自动关闭
<code>status</code>	TINYINT	DEFAULT 0	状态: 0-待审核, 1-招募中, 2-已匹配, 3-已完成, 4-已关闭
<code>is_anonymous</code>	BOOLEAN	DEFAULT FALSE	是否匿名发布, 保护隐私
<code>created_at</code>	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	发布时间

建表语句

```
-- 3. 需求发布表
CREATE TABLE `help_requests` (
  `request_id` BIGINT PRIMARY KEY AUTO_INCREMENT,
  `publisher_id` BIGINT NOT NULL COMMENT '对应student_user_id',
  `category_id` INT NOT NULL COMMENT '分类配置表ID',
  `title` VARCHAR(100) NOT NULL,
  `description` TEXT NOT NULL,
  `reward_amount` DECIMAL(10,2) DEFAULT 0.00,
  `deadline` DATETIME NOT NULL,
  `status` TINYINT DEFAULT 0 COMMENT '0:待审核, 1:招募中, 2:已匹配, 3:已完成, 4:已关闭',
  `is_anonymous` BOOLEAN DEFAULT FALSE,
  `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (`publisher_id`) REFERENCES `student_users`(`student_user_id`),
  INDEX `idx_status_category` (`status`, `category_id`),
```

```
INDEX `idx_deadline` (`deadline`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

4. 服务订单表 (`service_orders`)

字段名	类型	约束	说明
<code>order_id</code>	BIGINT	PRIMARY KEY, AUTO_INCREMENT	订单唯一标识
<code>request_id</code>	BIGINT	NOT NULL, UNIQUE, FOREIGN KEY	关联的原始需求ID，确保一个需求只产生一个订单
<code>demandor_id</code>	BIGINT	NOT NULL, FOREIGN KEY	需求方ID（等于 <code>help_requests.publisher_id</code> ）
<code>server_id</code>	BIGINT	NOT NULL, FOREIGN KEY	接单服务方ID
<code>current_state</code>	VARCHAR(50)	NOT NULL	状态模式中的状态类名，如 <code>PendingState</code> 、 <code>InProgressState</code> 等
<code>created_at</code>	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	订单创建时间（匹配成功时间）
<code>completed_at</code>	DATETIME		订单完成时间
<code>review_score</code>	TINYINT		评价分数，1~5星或1~10分，由需求方填写
<code>review_comment</code>	TEXT		评价文本
<code>has_complaint</code>	BOOLEAN	DEFAULT FALSE	是否发起过投诉
<code>complaint_reason</code>	TEXT		投诉原因（若有）

建表语句

```
-- 4. 服务订单表
CREATE TABLE `service_orders` (
  `order_id` BIGINT PRIMARY KEY AUTO_INCREMENT,
  `request_id` BIGINT NOT NULL UNIQUE,
  `demandor_id` BIGINT NOT NULL,
  `server_id` BIGINT NOT NULL,
  `current_state` VARCHAR(50) NOT NULL COMMENT '状态模式对应的状态类名',
  `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  `completed_at` DATETIME,
  `review_score` TINYINT,
  `review_comment` TEXT,
  `has_complaint` BOOLEAN DEFAULT FALSE,
  `complaint_reason` TEXT,
  FOREIGN KEY (`request_id`) REFERENCES `help_requests` (`request_id`),
```

```
FOREIGN KEY (`demander_id`) REFERENCES `student_users` (`student_user_id`),  
FOREIGN KEY (`server_id`) REFERENCES `student_users` (`student_user_id`),  
INDEX `idx_demander` (`demander_id`),  
INDEX `idx_server` (`server_id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```
